

Abstraction : Managing Different Media
S. Nisanth , ID = 12576

Code:

Program.cs:

```
class TuneIn{
    public static void Main(){
        Song song= new Song();
        Movie movie= new Movie();
        Podcast podcast= new Podcast();

        song.PlayMedia();
        song.PauseMedia();
        song.StopMedia();

        movie.PlayMedia();
        movie.PauseMedia();
        movie.StopMedia();

        podcast.PlayMedia();
        podcast.PauseMedia();
        podcast.StopMedia();

        Media media= new Song();

        media.IncreaseVolume();
        media.DecreaseVolume();

        Console.WriteLine("\nAll methods ave been called.");
    }
}
```

Movie.cs:

```
class Movie : Media
{
    public override void PlayMedia()
    {
        Console.WriteLine("\nPlaying Movie...");
    }

    public override void PauseMedia()
    {
        Console.WriteLine("Pausing Movie...");
    }

    public override void StopMedia()
    {
        Console.WriteLine("Stopping Movie...");
    }
}
```

```
}
```

Podcast.cs:

```
class Podcast : Media
{
    public override void PlayMedia()
    {
        Console.WriteLine("\nPlaying Podcast...");
    }

    public override void PauseMedia()
    {
        Console.WriteLine("Pausing Podcast...");
    }

    public override void StopMedia()
    {
        Console.WriteLine("Stopping Podcast...");
    }
}
```

Song.cs:

```
class Song : Media
{
    public override void PlayMedia()
    {
        Console.WriteLine("\nPlaying Song...");
    }

    public override void PauseMedia()
    {
        Console.WriteLine("Pausing Song...");
    }

    public override void StopMedia()
    {
        Console.WriteLine("Stopping Song...");
    }
}
```

Output:

(Next Page...)

```
PS C:\Users\nisanth.saravanan\Desktop\L0 Assesment> & 'c:\Users\nisanth.saravanan\.vscode\extensions\ms-dotnet  
tools.csharp-2.63.32-win32-x64\.debugger\x86_64\vsdbg.exe' '--interpreter=vscode' '--connection=6e0138b78d5c44b  
09bf0ac8e6359ddb9'
```

```
Playing Song...  
Pausing Song...  
Stopping Song...
```

```
Playing Movie...  
Pausing Movie...  
Stopping Movie...
```

```
Playing Podcast...  
Pausing Podcast...  
Stopping Podcast...
```

```
Increasing Volume...  
Decreasing Volume...
```

```
All methods ave been called.  
PS C:\Users\nisanth.saravanan\Desktop\L0 Assesment>
```

Inheritance - Different Types of Employees
S. Nisanth , ID = 12576

Code:

Employee.cs:

```
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double Salary { get; set; }
    public double Bonus = 10.00;

    public Employee(int id, string name, double salary)
    {
        Id = id;
        Name = name;
        Salary = salary;
    }

    public Employee()
    {
        return;
    }

    public double BonusAmount(List<Employee> employees)
    {
        Console.WriteLine("\nEnter the ID:");
        int id = int.Parse(Console.ReadLine());

        Employee employee = employees.FirstOrDefault(e => e.Id == id);
        if (employee == null)
        {
            return 0.00;
        }
        else
        {
            return employee.Salary * employee.Bonus * 0.10;
        }
    }

    public void AddEmployee(List<Employee> employees)
    {
        Console.WriteLine("\nEnter the Employee Details: ");
        Console.Write("ID: ");
        int id = int.Parse(Console.ReadLine());
        Console.Write("Name: ");
        string name = Console.ReadLine();
        Console.Write("Salary: ");
        double salary = double.Parse(Console.ReadLine());
    }
}
```

```

        Employee employee = new Employee(id, name, salary);
        employees.Add(employee);
    }

    public virtual void RemoveEmployee(List<Employee> employees)
    {
        Console.WriteLine("\nEnter the ID to remove: ");
        int id = int.Parse(Console.ReadLine());

        Employee employeeToRemove = employees.Find(x => x.Id == id);
        if (employeeToRemove != null)
        {
            employees.Remove(employeeToRemove);
            Console.WriteLine($"The employee of ID {id} removed successfully.");
        }
        else
        {
            Console.WriteLine($"The employee of ID {id} does not exist.");
        }
    }
}

```

Manager.cs:

```

class Manager : Employee
{
    public int NumberOfDirectReports { get; set; }

    public Manager(int id, string name, double salary, int numberOfDirectReports) : base(id,
name, salary)
    {
        NumberOfDirectReports = numberOfDirectReports;
    }

    public Manager(){
        return;
    }

    public double BonusAmount(List<Manager> managers)
    {
        Console.WriteLine("\nEnter the ID:");
        int id = int.Parse(Console.ReadLine());

        Manager manager = managers.FirstOrDefault(e => e.Id == id);
        if (manager == null)
        {
            return 0.00;
        }
        else
        {
            return manager.Salary * 0.10 + manager.NumberOfDirectReports * 100;
        }
    }
}

```

```

}

public void AddManager(List<Manager> managers)
{
    Console.WriteLine("\nEnter the Manager Details: ");
    Console.Write("ID: ");
    int id = int.Parse(Console.ReadLine());
    Console.Write("Name: ");
    string name = Console.ReadLine();
    Console.Write("Salary: ");
    double salary = double.Parse(Console.ReadLine());
    Console.Write("Number of direct reports: ");
    int numberOfDirectReports = int.Parse(Console.ReadLine());

    Manager manager = new Manager(id, name, salary, numberOfDirectReports);
    managers.Add(manager);
}

public virtual void RemoveManager(List<Manager> managers)
{
    Console.WriteLine("\nEnter the ID to remove: ");
    int id = int.Parse(Console.ReadLine());

    Manager ManagerToRemove = managers.Find(x => x.Id == id);
    if (ManagerToRemove != null)
    {
        managers.Remove(ManagerToRemove);
        Console.WriteLine($"The Manager of ID {id} removed successfully.");
    }
    else
    {
        Console.WriteLine($"The Manager of ID {id} does not exist.");
    }
}
}

```

Contractor.cs:

```

class Contractor : Employee
{
    public double HourlyRate { get; set; }
    public double HoursWorked { get; set; }

    public Contractor(int id, string name, double salary, double hourlyRate, double hoursWorked) : base(id, name, salary)
    {
        HourlyRate = hourlyRate;
        HoursWorked = hoursWorked;
    }

    public Contractor(){
        return;
    }
}

```

```

}

public double BonusAmount(List<Contractor> contractors)
{
    Console.WriteLine("\nEnter the ID:");
    int id = int.Parse(Console.ReadLine());

    Contractor contractor = contractors.FirstOrDefault(e => e.Id == id);
    if (contractor == null)
    {
        return 0.00;
    }
    else
    {
        return contractor.HourlyRate * contractor.HoursWorked * 0.05;
    }
}

public void AddContractor(List<Contractor> contractors)
{
    Console.WriteLine("\nEnter the Contractor Details: ");
    Console.Write("ID: ");
    int id = int.Parse(Console.ReadLine());
    Console.Write("Name: ");
    string name = Console.ReadLine();
    Console.Write("Salary: ");
    double salary = double.Parse(Console.ReadLine());
    Console.Write("Hourly Rate: ");
    double hourlyRate = double.Parse(Console.ReadLine());
    Console.Write("Hours Worked: ");
    double hoursWorked = double.Parse(Console.ReadLine());

    Contractor contractor = new Contractor(id, name, salary, hourlyRate, hoursWorked);
    contractors.Add(contractor);
}

public virtual void RemoveContractor(List<Contractor> Contractors)
{
    Console.WriteLine("\nEnter the ID to remove: ");
    int id = int.Parse(Console.ReadLine());

    Contractor ContractorToRemove = Contractors.Find(x => x.Id == id);
    if (ContractorToRemove != null)
    {
        Contractors.Remove(ContractorToRemove);
        Console.WriteLine($"The Contractor of ID {id} removed successfully.");
    }
    else
    {
        Console.WriteLine($"The Contractor of ID {id} does not exist.");
    }
}

```

```
}
```

Program.cs:

```
class Company
{
    public List<Employee> employees = new List<Employee>();
    public List<Manager> managers = new List<Manager>();
    public List<Contractor> contractors = new List<Contractor>();

    public static void Main()
    {
        Company company = new Company();
        Employee employee = new Employee();
        Manager manager = new Manager();
        Contractor contractor = new Contractor();

        checkpoint1:
        Console.WriteLine("\nEmployee Record System: ");
        Console.WriteLine("1. Add Employee.");
        Console.WriteLine("2. Add Manager.");
        Console.WriteLine("3. Add Contractor.");
        Console.WriteLine("4. Remove Employee.");
        Console.WriteLine("5. Calculate Bonus.");
        Console.WriteLine("0. Exit.");

        Console.WriteLine("\nEnter your choice: ");
        int choice1 = int.Parse(Console.ReadLine());

        switch (choice1)
        {
            default:
            {
                Console.WriteLine("\nINVALID CHOICE! TRY AGAIN!");
                goto checkpoint1;
            }

            case 0:
            {
                Console.WriteLine("\nLOGGING OUT...");
                return;
            }

            case 1:
            {
                employee.AddEmployee(company.employees);
                goto checkpoint1;
            }

            case 2:
            {
                manager.AddManager(company.managers);
```



```

        goto checkpoint1;
    }

    case 3:
    {
        contractor.AddContractor(company.contractors);
        goto checkpoint1;
    }

    case 4:
    {
        goto removal;
    }

    case 5:
    {
        goto bonuses;
    }
}

```

removal:

```

Console.WriteLine("\nChoose what to remove:");
Console.WriteLine("1. Employee.");
Console.WriteLine("2. Manager.");
Console.WriteLine("3. Contractor.");
Console.WriteLine("0. Back.");

Console.WriteLine("\nEnter your choice: ");
int choice2 = int.Parse(Console.ReadLine());

switch (choice2)
{
    default:
    {
        Console.WriteLine("\nINVALID CHOICE! TRY AGAIN!");
        goto removal;
    }

    case 0:
    {
        goto checkpoint1;
    }

    case 1:
    {
        employee.RemoveEmployee(company.employees);
        goto checkpoint1;
    }

    case 2:
    {
        manager.RemoveManager(company.managers);
    }
}

```

```

        goto checkpoint1;
    }

    case 3:
    {
        contractor.RemoveContractor(company.contractors);
        goto checkpoint1;
    }
}

```

bonuses:

```

Console.WriteLine("\nChoose for whom to calculate:");
Console.WriteLine("1. Employee.");
Console.WriteLine("2. Manager.");
Console.WriteLine("3. Contractor.");
Console.WriteLine("0. Back.");

```

```

Console.WriteLine("\nEnter your choice: ");
int choice3 = int.Parse(Console.ReadLine());

```

```

switch (choice3)
{
    default:
    {
        Console.WriteLine("\nINVALID CHOICE! TRY AGAIN!");
        goto bonuses;
    }
}

```

```

    case 0:
    {
        goto checkpoint1;
    }
}

```

```

    case 1:
    {
        Console.WriteLine($"The bonus of the selected regular employee is
{employee.BonusAmount(company.employees)}");
        goto checkpoint1;
    }
}

```

```

    case 2:
    {
        Console.WriteLine($"The bonus of the selected manager is
{manager.BonusAmount(company.managers)}");
        goto checkpoint1;
    }
}

```

```

    case 3:
    {
        Console.WriteLine($"The bonus of the selected contractor is
{contractor.BonusAmount(company.contractors)}");
    }
}

```

```

        goto checkpoint1;
    }
}
}
}

```

Output:

```

PS C:\Users\nisanth.saravanan\Desktop\L0 Assesment> & 'c:\Users\nisanth.saravanan\.vscode\extensions\ms-dotnet
tools.csharp-2.63.32-win32-x64\.debugger\x86_64\vsdbg.exe' '--interpreter=vscode' '--connection=0ef72ffad08c45d
691bea333f5da9dc8'

```

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

1

Enter the Employee Details:

ID: 1

Name: Charles

Salary: 50000

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

3

Enter the Contractor Details:

ID: 3

Name: Juan

Salary: 12000

Hourly Rate: 1000

Hours Worked: 12

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

2

Enter the Manager Details:

ID: 2

Name: Kenny

Salary: 75000

Number of direct reports: 100

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

5

Choose for whom to calculate:

1. Employee.
2. Manager.
3. Contractor.
0. Back.

Enter your choice:

1

Enter the ID:

1

The bonus of the selected regular employee is 50000.

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

5

Choose for whom to calculate:

1. Employee.
2. Manager.
3. Contractor.
0. Back.

Enter your choice:

2

Enter the ID:

2

The bonus of the selected manager is 13000

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

5

Choose for whom to calculate:

1. Employee.
2. Manager.
3. Contractor.
0. Back.

Enter your choice:

3

Enter the ID:

3

The bonus of the selected contractor is 72

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

4

Choose what to remove:

1. Employee.
2. Manager.
3. Contractor.
0. Back.

Enter your choice:

1

Enter the ID to remove: 1

The employee of ID 1 removed successfully.

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

4

Choose what to remove:

1. Employee.
2. Manager.
3. Contractor.
0. Back.

Enter your choice:

2

Enter the ID to remove: 2

The Manager of ID 2 removed successfully.

Employee Record System:

1. Add Employee.
2. Add Manager.
3. Add Contractor.
4. Remove Employee.
5. Calculate Bonus.
0. Exit.

Enter your choice:

0

LOGGING OUT...

PS C:\Users\nisanth.saravanan\Desktop\L0 Assesment> █